

LIVESTREAM OPERATIONAL INTELLIGENCE INTEGRATION

Date: 2026-06-02

OBJECTIVE

Transform livestreaming from a Socket.IO-only feature into an Operational Intelligence data source for YenkaSAI.

Required architecture:

Livestream Client -> Socket Layer -> Livestream Event Service -> Operational Events Layer (OIL) -> YenkaSAI

FINDINGS

1. Livestream Socket.IO sync already existed, but events were emitted as older lowercase YME events such as 'live_comment', 'live_reaction', 'live_joined', and 'gift_sent'.
2. The current real-time path preserved UX but did not consistently expose canonical platform event types such as 'STREAM_COMMENT', 'STREAM_GIFT', 'STREAM_REPORT', or 'STREAM_VIEW_DURATION'.
3. HTTP livestream routes handled important actions outside the socket path, especially join-token requests, host end, auto-end, viewer leave, and gifts.
4. The AI event relay was configured for '/api/events', while the requested OIL ingestion target is '/api/events/ingest'.
5. YME scoring, consolidation, interest extraction, and recommendation signals did not know the new stream event names.

FIXES IMPLEMENTED

1. Canonical Livestream Operational Event Service

Created 'src/services/livestream/operationalEvents.service.js'.

This service defines all required event types:

- 'STREAM_STARTED'
- 'STREAM_ENDED'
- 'STREAM_JOINED'
- 'STREAM_LEFT'
- 'STREAM_COMMENT'
- 'STREAM_LIKE'
- 'STREAM_GIFT'
- 'STREAM_SHARE'
- 'STREAM_REPORT'
- 'STREAM_FOLLOW_HOST'
- 'STREAM_PIN_COMMENT'
- 'STREAM_MODERATION_ACTION'
- 'STREAM_BAN_USER'
- 'STREAM_WARNING'

- 'STREAM_VIEW_DURATION'
- 'STREAM_PEAK_VIEWERS'

It emits the canonical schema:

```
{
  "eventType": "STREAM_COMMENT",
  "streamId": "...",
  "userId": "...",
  "hostId": "...",
  "timestamp": "...",
  "metadata": {}
}
```

Evidence:

- Event type constants: 'src/services/livestream/operationalEvents.service.js:4'
- Schema builder: 'src/services/livestream/operationalEvents.service.js:163'
- OIL/YME publishing path: 'src/services/livestream/operationalEvents.service.js:284'

2. Analytics Metrics

The operational event service now populates livestream metrics:

- 'totalViewers'
- 'concurrentViewers'
- 'peakViewers'
- 'commentsPerMinute'
- 'likesPerMinute'
- 'giftsPerMinute'
- 'watchTime'
- 'averageWatchTime'
- 'retentionRate'
- 'giftRevenue'
- 'topGifters'
- 'topStreams'

Evidence:

- Metrics state and public metrics: 'src/services/livestream/operationalEvents.service.js:46'
- Metrics update logic: 'src/services/livestream/operationalEvents.service.js:98'
- Top streams accessor: 'src/services/livestream/operationalEvents.service.js:335'
- API metrics endpoints: 'routes/livestream.routes.js:449'

3. Moderation Processing

Livestream events now generate moderation signals for:

- spam comments
- gift fraud
- bot viewers
- abuse reports
- suspicious engagement spikes

Detected signals create 'ModerationItem' records with 'source: livestream_operational_intelligence'.

Evidence:

- Moderation signal detection: 'src/services/livestream/operationalEvents.service.js:185'
- Moderation queue item creation: 'src/services/livestream/operationalEvents.service.js:244'

4. Socket.IO Room Preservation and New Broadcasts

Socket.IO rooms now include the required room shape:

- 'stream:{streamId}'

Compatibility rooms remain active:

- 'livestream_{streamId}'
- 'live:{streamId}'

Real-time broadcasts now include the requested names:

- 'new_comment'
- 'new_like'
- 'new_gift'
- 'viewer_joined'
- 'viewer_left'
- 'gift_animation'

Evidence:

- Room update: 'src/services/livestream/livestream.service.js'
- 'new_comment': 'src/services/livestream/livestream.socket.js:412'
- 'new_like': 'src/services/livestream/livestream.socket.js:437'
- 'viewer_joined': 'src/services/livestream/livestream.socket.js:313'
- 'viewer_left': 'src/services/livestream/livestream.socket.js:356'
- 'new_gift' and 'gift_animation': 'routes/livestream.routes.js:757'

5. Socket Layer Event Coverage

Socket handlers now emit operational events for:

- start
- join

- leave
- comment
- like
- share
- report
- follow host
- pin comment
- moderation action
- ban user
- warning
- view duration
- disconnect watch duration

Evidence:

- Shared socket operational emitter: 'src/services/livestream/livestream.socket.js:10'
- Comment and like mapping: 'src/services/livestream/livestream.socket.js:394'
- Share, report, follow-host, pin-comment, moderation actions: 'src/services/livestream/livestream.socket.js:445'
- Disconnect leave and view duration capture: 'src/services/livestream/livestream.socket.js:853'

6. HTTP Route Event Coverage

HTTP routes now emit operational events for:

- livestream join token
- host end
- auto-end
- startup failure
- leave
- watch duration
- gift

Evidence:

- HTTP operational emitter: 'routes/livestream.routes.js:171'
- Join token event: 'routes/livestream.routes.js:558'
- End event: 'routes/livestream.routes.js:621'
- Leave and view-duration event: 'routes/livestream.routes.js:659'
- Gift event: 'routes/livestream.routes.js:773'

7. OIL and YenkaSAI Relay

The intelligence relay default ingest path now targets:

`/api/events/ingest`

The relay supports all canonical livestream events as normalized 'stream_*' operational events and forwards livestream metadata including stream ID, host ID, metrics, moderation

signals, and recommendation signals.

Evidence:

- Default ingest path: 'src/intelligence/services/eventPublisher.service.js:8'
- Supported stream event types: 'src/intelligence/services/eventPublisher.service.js:54'
- Livestream relay metadata mapping:
'src/intelligence/services/eventPublisher.service.js:576'

8. YME Memory and Recommendation Integration

YME now recognizes stream event types in:

- source app config
- event ingestion aliases
- importance scoring
- interest extraction
- recommendation scoring
- consolidation trigger list

Evidence:

- Source apps and event types: 'src/yme/config/yme.config.js'
- Event aliases: 'src/yme/services/eventIngestion.service.js'
- Importance scoring: 'src/yme/services/importanceScoring.service.js'
- Interest weights: 'src/yme/services/interestExtraction.service.js'
- Recommendation weights: 'src/yme/services/recommendationSignals.service.js'
- Consolidation triggers: 'src/yme/services/consolidation.service.js'

VERIFICATION EVIDENCE

Backend Verification

Command:

npm test

Result:

```
tests 10
pass 10
fail 0
duration_ms 10022.635969
```

Latest rerun after frontend blocker cleanup:

tests 10
pass 10
fail 0
duration_ms 1733.445269

Latest rerun after adding the OIL HTTP ingest route:

tests 12
pass 12
fail 0
duration_ms 1866.716462

Additional syntax checks passed:

```
node --check src/services/livestream/livestream.socket.js
node --check routes/livestream.routes.js
node --check routes/events.routes.js
node --check src/config/apiRoutes.js
node --check src/services/livestream/operationalEvents.service.js
node --check src/intelligence/services/eventPublisher.service.js
```

Focused test evidence:

- Canonical livestream schema test: 'tests/livestreamOperationalEvents.test.js:11'
- Metrics population test: 'tests/livestreamOperationalEvents.test.js:33'
- AI relay 'STREAM_GIFT' mapping test: 'tests/intelligenceEventPublisher.test.js'
- OIL HTTP ingest route mapping test: 'tests/operationalEventsIngestRoute.test.js'

OIL Ingest Route Fix

Verification found that the production API currently returns '404' for:

```
POST /api/events/ingest
{"error": "API route not found"}
```

The local backend has now been fixed to mount a real OIL ingest endpoint:

```
POST /api/events/ingest
```

The endpoint accepts either a single event payload or '{ "events": [...] }', maps the

canonical livestream schema into YME ingest fields, and supports internal service authentication through 'YENKASA_EVENTS_INGEST_API_KEY', 'INTERNAL_PLATFORM_API_KEY', or 'LOG_INGEST_API_KEY'. If no valid internal key is supplied, it falls back to the existing bearer-token auth middleware.

Frontend Wiring Verification

The Android livestream screen was checked and patched against the backend socket API:

- Frontend listens to 'live_comment', 'live_join', 'live_leave', 'live_reaction', 'live_gift', 'live_room_joined', and 'live_ended'.
- Frontend now also listens to required platform broadcast aliases: 'new_comment', 'new_like', 'new_gift', 'viewer_joined', 'viewer_left', and 'gift_animation'.
- Frontend emits 'live_join', 'live_comment', 'live_reaction', 'live_leave', 'live_request_guest_seat', 'live_mute_guest', 'live_kick_guest', 'live_approve_guest_seat', and 'live_decline_guest_seat'.
- Backend preserved the legacy event names and emits the requested new broadcasts, so existing client UX remains wired while OIL events are emitted in parallel.
- The activity keeps 'clientEventId' de-duplication, so listening to legacy and new aliases does not double-render mirrored events.

Evidence:

- Frontend listeners and emitters: 'app/src/main/java/xyz/yenkasa/app/ui/LiveStreamActivity.kt'
- Backend legacy + new broadcasts: 'src/services/livestream/livestream.socket.js'
- Gift broadcast compatibility: 'routes/livestream.routes.js'

Android build verification:

```
./gradlew :app:testDebugUnitTest  
./gradlew :app:assembleDebug
```

Results:

```
:app:testDebugUnitTest BUILD SUCCESSFUL  
:app:assembleDebug BUILD SUCCESSFUL
```

Latest rerun after the new frontend alias wiring:

```
./gradlew :app:testDebugUnitTest  
BUILD SUCCESSFUL in 3m 11s  
31 actionable tasks: 8 executed, 23 up-to-date
```

The debug APK was generated at:

/Users/kofibright/yenkasaChat/app/build/outputs/apk/debug/app-debug.apk

Device install/runtime evidence:

adb devices

returned:

146353755V007782 device

The debug build was installed on the connected Android device:

```
./gradlew :app:installDebug  
Installed on 1 device.  
BUILD SUCCESSFUL in 1m 14s
```

The installed package was verified:

```
package: xyz.yenkasa.app  
launcher: xyz.yenkasa.app/.ui.SplashActivity  
device: TECNO BG6m - Android 14  
versionName: 0.5.2  
debuggable: true
```

ADB launch succeeded:

Starting: Intent { cmp=xyz.yenkasa.app/.ui.SplashActivity }

Runtime logcat showed the app stayed alive and routed to 'LoginActivity', with no 'FATAL EXCEPTION' or 'AndroidRuntime' crash for the launched app. Required livestream runtime permissions were granted by ADB before launch:

```
android.permission.CAMERA  
android.permission.RECORD_AUDIO  
android.permission.POST_NOTIFICATIONS
```

Endpoint auth check:

```
curl -i https://yenkasa-8rjea.ondigitalocean.app/api/livestream/active
```

returned:

HTTP/2 401

```
{"success":false,"message":"Access denied. Authorization header missing."}
```

Route verification confirms all livestream REST endpoints require 'auth':

```
POST /api/livestream/create
GET /api/livestream/active
GET /api/livestream/top-streams
GET /api/livestream/:id/metrics
GET /api/livestream/:id/creator-intelligence
POST /api/livestream/join/:id
POST /api/livestream/end/:id
POST /api/livestream/leave/:id
POST /api/livestream/gift
```

The connected device was not logged into Yenkasa after installation, so authenticated create/join/end/leave/gift calls could not be completed from the app. Only one device was connected, so a true host-screen plus joined-screen synchronization test could not be proven end to end in this run. The frontend is now wired for both host and joined screens to receive all required socket event names, and the backend tests verify the OIL event pipeline and metrics behavior.

Frontend Build Blockers Fixed

The Android app initially could not be tested because committed merge-conflict markers existed in resource and Kotlin files unrelated to the livestream backend integration. These blocked 'mergeDebugResources' and Kotlin compilation.

Fixed files:

- 'app/src/main/res/values/strings.xml'
- 'app/src/main/res/layout/activity_user_notifications.xml'
- 'app/src/main/java/xyz/yenkasa/app/adapter/NotificationAdapter.kt'
- 'app/src/main/java/xyz/yenkasa/app/model/NotificationModel.kt'
- 'app/src/main/java/xyz/yenkasa/app/network/ApiService.kt'
- 'app/src/main/java/xyz/yenkasa/app/ui/LiveStreamActivity.kt'

- 'app/src/main/java/xyz/yenkasa/app/ui/UserNotificationsActivity.kt'
- 'app/src/main/java/xyz/yenkasa/app/ui/PostActivity.kt'

After cleanup, Android resources, Kotlin compilation, unit tests, and debug APK assembly passed.

IMPLEMENTATION NOTES

The new metrics store is in-process. It is suitable for immediate operational intelligence and creator dashboard snapshots, but production-grade historical analytics should later persist rollups into MongoDB or a streaming analytics store.

The implementation preserves current client compatibility by keeping existing 'live_*' broadcasts while adding the requested event names.

CHANGED FILES

- 'src/services/livestream/operationalEvents.service.js'
- 'src/services/livestream/livestream.socket.js'
- 'src/services/livestream/livestream.service.js'
- 'routes/livestream.routes.js'
- 'routes/events.routes.js'
- 'src/config/apiRoutes.js'
- 'app/src/main/java/xyz/yenkasa/app/ui/LiveStreamActivity.kt'
- 'src/intelligence/services/eventPublisher.service.js'
- 'src/yme/config/yme.config.js'
- 'src/yme/services/eventIngestion.service.js'
- 'src/yme/services/importanceScoring.service.js'
- 'src/yme/services/interestExtraction.service.js'
- 'src/yme/services/recommendationSignals.service.js'
- 'src/yme/services/consolidation.service.js'
- 'tests/livestreamOperationalEvents.test.js'
- 'tests/intelligenceEventPublisher.test.js'
- 'tests/operationalEventsIngestRoute.test.js'

CONCLUSION

Livestreaming is now wired as an Operational Intelligence source. Real-time synchronization remains intact, and every major livestream action can now feed analytics, moderation, recommendations, YME memory, security detection, and the YenkaSAI event relay.